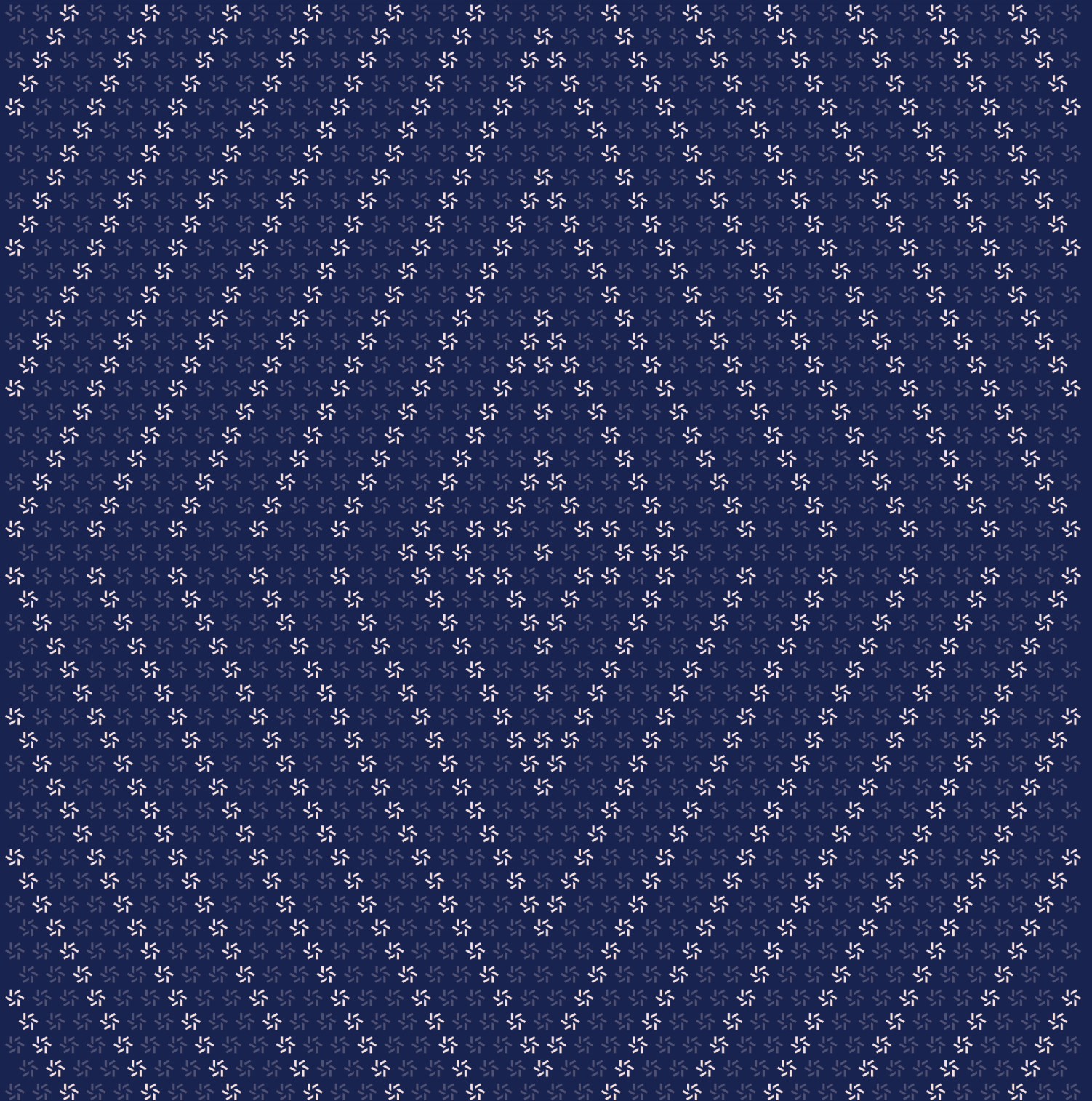


August 18, 2026

Tydro - Payload/Economic Review

Smart Contract Security Assessment



Contents

About Zellic	3
1. Overview	4
1.1. Executive Summary	4
1.2. Non-goals and Limitations	4
1.3. Results	4
2. Introduction	6
2.1. About Tydro - Payload/Economic Review	6
2.2. Methodology	6
2.3. Scope	8
2.4. Project Overview	8
2.5. Project Timeline	9
3. Discussion	10
3.1. Parameter change review	10
4. Assessment Results	12
4.1. Disclaimer	12

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and follow @zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a best-effort security assessment for Tydro on July 16th, 2026. During this engagement, Zellic reviewed Tydro's code for security vulnerabilities, design issues, and general weaknesses in security posture. This review was isolated to whether the code and configuration changes in [PR #9](#) match the intended result mentioned in the PR description, whether those changes introduce adverse effects, and whether the selected parameter values are locally sound.

1.2. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody
- Changes within the PR outside of scope

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.3. Results

During our assessment on the scoped Tydro - Payload/Economic Review contracts, there were no security vulnerabilities discovered.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Tydro in the Discussion section ([3.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	0

2. Introduction

2.1. About Tydro - Payload/Economic Review

Tydro contributed the following description of Tydro - Payload/Economic Review:

This payload separates the shared Tydro market into explicit borrowing corridors. Its primary objective is to make kBTC the main collateral for USDC debt while preventing kBTC and other collateral from using unrelated eMode debt markets.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion ([3.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Tydro - Payload/Economic Review Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only commits between 8fc7232...c1257e5
Repository	https://github.com/tydrohq/tydro-proposals-v3/pull/9/commits ↗
Version	c1257e58a1f6ade0f773ce1cf4486b885c42fd00
Programs	Parameter change review

2.4. Project Overview

Zellic was contracted to perform a best-effort security assessment for a total of 0.5 person-days. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Qingying Jie
✈ Engineer
qingying@zellic.io ↗

Kuilin Li
✈ Engineer
kuilin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 16, 2026 Start of primary review period

July 16, 2026 End of primary review period

3. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

3.1. Parameter change review

We were asked to perform a best-effort review of the parameter changes in [PR #9](#) as part of our engagement. In particular, this review focused on whether the code and configuration changes in [PR #9](#) match the intended result mentioned in the PR description, whether those changes introduce adverse effects, and whether the selected parameter values are locally sound.

At the time of our review, this pull request included seven commits:

- [8fc72322](#)
- [53ddebce](#)
- [79985d27](#)
- [87a5b65e](#)
- [d3554d05](#)
- [1f616051](#)
- [c1257e58](#)

We did not assess the following areas that were outside the scope of this separate portion of the engagement:

- Whether the proposed values are optimal from a business, governance, or market-strategy perspective
- Any second-order ecosystem effects, such as how the updated values may interact with broader Aave market conditions, liquidity, incentives, or future governance actions
- Other changes in [PR #9](#) outside the market configuration update described below, including changes to infrastructure, configuration, and tests relating to the project

[PR #9](#) updates Aave v3 Ink Whitelabel market configuration through the contract `AaveV3InkWhitelabel_InkMarketIsolationUpdate_20260622`. The parameter changes in [PR #9](#) can be summarized as follows:

- The USDC interest rate strategy is updated to an optimal utilization ratio of 95%, a base variable borrow rate of 6.5%, and zero values for both variable-rate slopes. This implements a flat borrow rate.
- WETH, USDT0, weETH, ezETH, and SolvBTC have their LTV set to 0 while keeping their existing liquidation thresholds, liquidation bonuses, and liquidation protocol fee unchanged. kBTC remains collateral-enabled, and its liquidation protocol fee remains unchanged, while its LTV changed from 72% to 85%, liquidation threshold changed from 77% to 87%, and liquidation bonus changed from 7.5% to 5%. The other collateral assets, namely sUSDe, USDC, USDe, syrupUSDT, wrsETH, USDG, and GHO, all retain an LTV of 0. Therefore, outside of eMode, kBTC is the only asset that can be used as

collateral.

- Borrowing is disabled for WETH, kBTC, USDT0, USDG, GHO, and USDe.
- Existing eMode categories 1 through 4 are all marked as isolated.
 - The other configuration of eMode category 1 (LRT Main) remains unchanged: weETH and ezETH can be used as collateral to borrow WETH. wrsETH is also a collateral asset in this eMode, but it is marked as ltvzero.
 - All collateral assets (sUSDe and USDe) in eMode category 2 (sUSDe_USDe__USDT0_USDC_USDG) are marked as ltvzero.
 - The only collateral asset (USDe) in eMode category 3 (USDe__USDT0_USDC_USDG) is marked as ltvzero.
 - The other configuration of eMode category 4 (syrupUSDT__USDT0) remains unchanged: syrupUSDT can be used as collateral to borrow USDT0.
- Two new isolated eMode categories are created:
 - sUSDe and USDe as collateral against USDT0 and USDG with parameters of 90% LTV, 92% liquidation threshold, and 4% liquidation bonus (note that these parameter values are consistent with those of eMode category 2), and
 - SolvBTC as collateral against GHO with parameters of 70% LTV, 72% liquidation threshold, and 7.5% liquidation bonus.

The above changes are fully consistent with the core intent described in the PR description. The contract implementation also fully follows the standard Aave V3 config-engine payload pattern and uses the same categories of update hooks as the reference mock examples in [the Aave V3 config engine mocks](#).

4. Assessment Results

During our assessment on the scoped Tydro - Payload/Economic Review contracts, there were no security vulnerabilities discovered.

4.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.